Software Deployment Plan

Group members:

- Boon
- Ryuki
- Pepper
- Thiri

1. System Requirements

Supported OS (runtime & test targets)

- Windows: Windows 10/11 (64-bit)
- macOS: Latest two versions (e.g., Sonoma, Sequoia)
- Linux: Ubuntu 20.04+ (also supports 18.04+ if Python 3.8 is available), Ubuntu 14+ (legacy), openSUSE Leap 15.x

Hardware

- CPU: x86_64
- RAM: 2 GB
- Disk: 1 GB free for application

Software

1. Containerized (recommended): Docker 24+, Docker Compose 2+
2. Native/venv: Python 3.10–3.12, pip, a C compiler toolchain for any package builds

Dependencies handled by deployment

- Python packages from requirements.txt (generated as part of deployment)
- SQLite (bundled with Python), app DB file app.db or migrations
- Optional: Nginx (production reverse proxy), systemd unit (Linux) or Windows Service (NSSM)

2. Deployment Strategy Summary

Docker

- Run the app in a Docker container with Gunicorn and `main.py`. Use a volume for `app.db` and a `.env` file for secrets. You can add Nginx for HTTPS if needed.

Zip/Tarball:

- Share a zip file with the code, `requirements.txt`, and setup scripts. It installs in a virtual environment and runs with Gunicorn or Waitress.

Both paths create the DB (SQLite) on first run or run a migration/seed step. Artifacts are versioned by git tag (e.g., v1.0.0) and released on GitHub Releases.

3. Installation Package Contents

3.1 Required source/compiled files

- App modules: main.py, database.py, matching.py, message.py, note.py, user_repo.py, utility.py GitHub
- WSGI Entry Point: main:app
- requirements.txt

3.2 Required third-party components

- Python libraries listed in requirements.txt
- Docker Engine and Docker Compose

3.3 Required graphical assets/config/other non-program files

- static/ folder for images, styles, and other assets
- .env.example template for environment variables

3.4 Documentation files to be provided

- README_DEPLOY.md (how to run both paths)
- CHANGELOG.md (version notes)

- ARCHITECTURE.md (brief component diagram & data flow)
- OPERATIONS.md (backup/restore, logs, rotation)

3.5 Development files/components to be excluded

- __pycache__/, .pytest_cache/, .venv/
- Local .env (never commit), test datasets, notebooks
- Developer-only scripts not needed in prod

4. Additional Code Required for Deployment

- Dockerfile: multistage build (pip install, nonroot user)
- docker-compose.yml: web service + volume for /data/app.db
- Start scripts: run_gunicorn.sh (Linux/macOS), run_waitress.ps1 (Windows)
- Install scripts: setup.sh, setup.ps1 (create venv, install reqs, create .env)
- DB migration/seed: manage.py or seed.py (init DB, apply schema, seed demo users)
- Healthcheck: /healthz endpoint for container health
- Nginx config (optional): nginx.conf with reverse proxy → web:8000, TLS placeholders

5. Deployment Tasks (Checklist)

1. Bump version in __init__.py (or config) and CHANGELOG.md.
2. Freeze dependencies: pip install -r requirements.in && pip freeze > requirements.txt.
3. Run full test suite & lints.
4. Sanity check .env.example has all required keys.
5. Ensure DB migrations/seed scripts up to date
6. Container: docker build -t ghcr.io/<org>/sen201-dating:<tag> .
7. Compose: confirm docker-compose.yml references image tag & volume mapping.
8. Zip Release: git archive -o sen201-dating-<tag>.zip --prefix=app/ <tag> and add scripts + docs.
9. Deployment Test Plan

Goal: Verify installation, configuration, database, and core user flows on each OS; ensure idempotent re-deploys.

6. Deployment Test Plan

1. Smoke/Health checks: app starts cleanly, `/healthz` returns 200.
2. Configuration tests: `.env` handling, port binding.
3. Database tests: initialization, seeding, idempotence.
4. Core user flows: register/login, profile CRUD, matching, messaging, notes.

5. Cross-platform tests: Windows/macOS/Linux with browser coverage.
6. Security & resilience: secrets, rate-limiting, persistence.
7. Acceptance criteria: all tests pass, fresh install < 10 min, data preserved on redeploy.